Lecture 7: Advanced Schema Design & Normalization

Agenda: From Theory to Practice

1. **Recap & The Big Picture:** Connecting Lectures 4, 5, and 6
2. **A Comprehensive Example:** From E/R Diagram to Normalization
3. **Boyce-Codd Normal Form (BCNF):** The "Pure" Normal Form
4. **Decomposition in Practice:** Lossless Joins vs. Dependency Preservation
5. **Third Normal Form (3NF):** The Pragmatic Compromise
6. **Beyond FDs:** Multivalued Dependencies (MVDs) & 4NF

Database Design

The Blueprint

Conceptual Design

- E/R Diagrams
- Entities & Relationships
- Constraints

The Rules

Logical Design Theory

- Functional Dependencies
- Keys (Super, Candidate)
- Closures & Armstrong's Axioms

The Refinement

Normalization

- BCNF, 3NF, 4NF
- Decomposition
- Anomaly Removal

The Goal

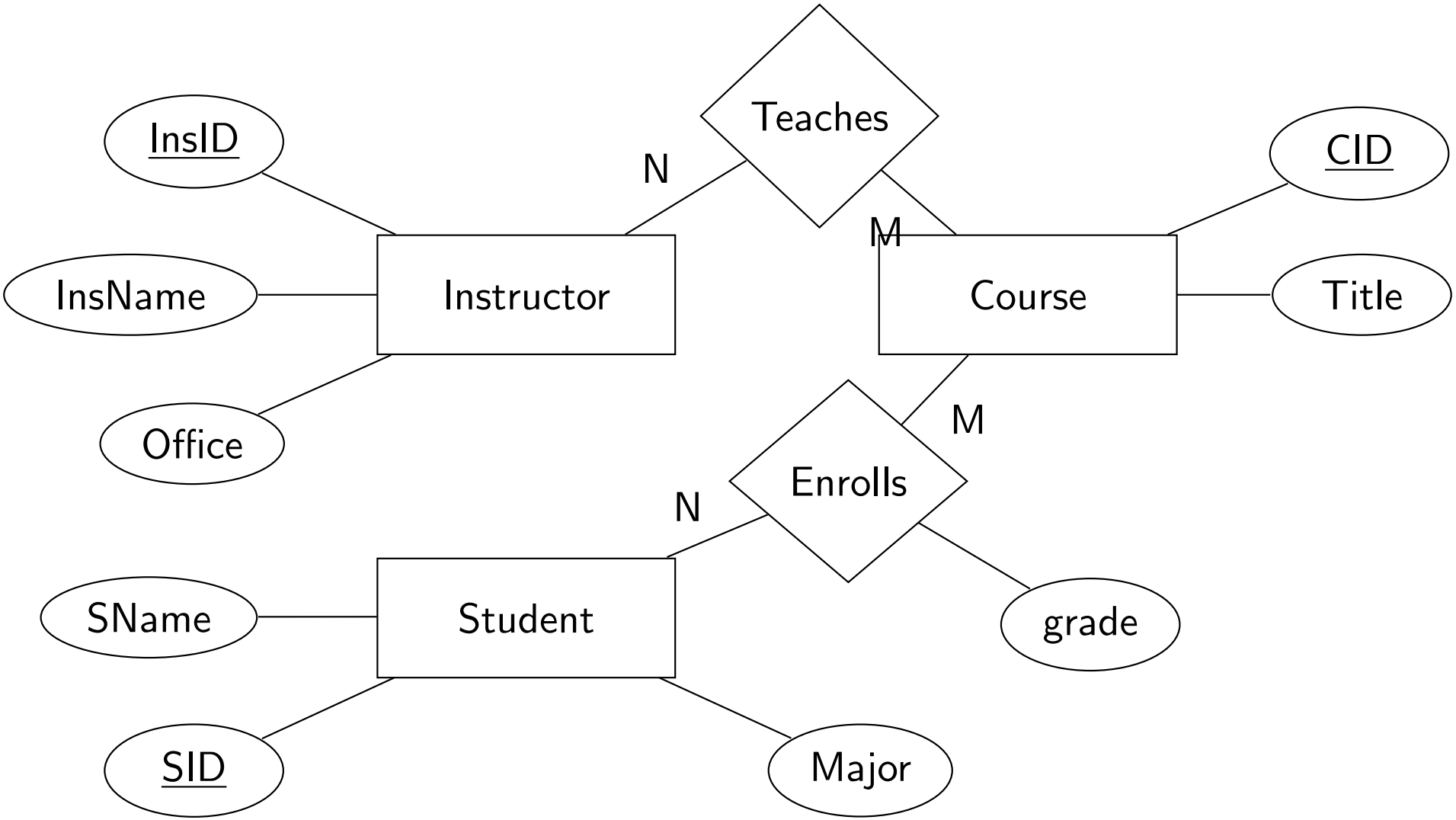
To transform high-level requirements into a robust, efficient, and anomaly-free relational schema. Today, we put all the pieces together.

Example: Course Management

We need to design a database for a university to manage course enrollments. Here are the business rules we gathered during requirements analysis:

- We track **Instructors** (identified by an 'InsID', they have a 'name' and 'office' number).
- We track **Courses** (identified by a 'CID', they have a 'title').
- We track **Students** (identified by a 'SID', they have a 'name' and a 'major').
- An instructor can teach multiple courses, and a course can be taught by multiple instructors (a teaching team).
- A student can enroll in multiple courses.
- For each course a student takes, we record their final 'grade'.
- Each instructor has a single designated 'office', and each office is assigned to only one instructor.

Step 1: The E/R Diagram



Step 2: The "Universal" Relation Schema

Initial Schema Translation

A naive translation of this E/R diagram might lead us to a single, large relation that captures everything. This is a common starting point for analysis, but as we'll see, it's a terrible final design.

Step 2: The "Universal" Relation Schema

Initial Schema Translation

A naive translation of this E/R diagram might lead us to a single, large relation that captures everything. This is a common starting point for analysis, but as we'll see, it's a terrible final design.

Universal Relation R

$R = (\underline{\text{InsID}}, \text{InsName}, \text{Office}, \underline{\text{CID}}, \text{Title}, \underline{\text{SID}}, \text{SName}, \text{Major}, \text{Grade})$

Step 2: The "Universal" Relation Schema

Initial Schema Translation

A naive translation of this E/R diagram might lead us to a single, large relation that captures everything. This is a common starting point for analysis, but as we'll see, it's a terrible final design.

Universal Relation R

$R = (\underline{\text{InsID}}, \text{InsName}, \text{Office}, \underline{\text{CID}}, \text{Title}, \underline{\text{SID}}, \text{SName}, \text{Major}, \text{Grade})$

Problem

This schema is riddled with redundancy and is vulnerable to update, insertion, and deletion anomalies. Why? Let's use the tools from previous lectures to find out.

Step 3: Finding Functional Dependencies

Deriving FDs from Business Rules

Based on our scenario, we can state the following FDs:

- An instructor's ID determines their name and office:
  $F_1 : InsID \rightarrow InsName, Office$
- An office is assigned to only one instructor:
  $F_2 : Office \rightarrow InsID$
- A course ID determines its title:
  $F_3 : CID \rightarrow Title$
- A student ID determines their name and major:
  $F_4 : SID \rightarrow SName, Major$
- A student gets one grade for a given course:
  $F_5 : SID, CID \rightarrow Grade$

Let $F = \{F_1, F_2, F_3, F_4, F_5\}$.

Step 4: Finding The Key

Let's find the candidate key(s) for our universal relation:
R = (InsID, InsName, Office, CID, Title, SID, SName, Major, Grade)

Attribute Closure Algorithm

We need to find a set of attributes X such that $X^+ = R$.

- Start with attributes that never appear on the right side of any FD. In our set F, every attribute appears on the right side of some FD or can be derived. This means we must combine attributes.
- Let's try combining the "ID" attributes that correspond to our many-to-many relationships: {InsID, CID, SID}.

Step 4: Finding The Key

Attribute Closure Algorithm-continued

- Calculate $\{InsID, CID, SID\}^+$:
  1. Start with $\{InsID, CID, SID\}$.
  2. From $InsID \rightarrow InsName, Office$, add InsName, Office. Result: $\{InsID, CID, SID, InsName, Office\}$.
  3. From $CID \rightarrow Title$, add Title. Result: $\{InsID, CID, SID, InsName, Office, Title\}$.
  4. From $SID \rightarrow SName, Major$, add SName, Major. Result: $\{InsID, CID, SID, InsName, Office, Title, SName, Major\}$.
  5. From $SID, CID \rightarrow Grade$, add Grade. Result: $\{InsID, CID, SID, InsName, Office, Title, SName, Major, Grade\}$.

Conclusion

The set $\{InsID, CID, SID\}$ determines all other attributes. Therefore, the primary key for this universal relation is {**InsID, CID, SID**}.

Boyce-Codd Normal Form (BCNF)

Definition

A relation R is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$ that holds on R, X must be a **superkey** of R.

Boyce-Codd Normal Form (BCNF)

Definition

A relation R is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$ that holds on R, X must be a **superkey** of R.

The "Purity" Test

In simpler terms: "Every determinant must be a superkey."

- A determinant is the left-hand side of an FD.
- It means that the only things that can determine other attributes are keys that can determine *all* attributes.
- This is a very strict condition, which is why BCNF is considered the "purest" normal form based on FDs.

Is Our University Relation in BCNF?

Relation: R = (InsID, InsName, Office, CID, Title, SID, SName, Major, Grade)
Key: {InsID, CID, SID}

The Test

For every FD $X \rightarrow Y$, is X a superkey of R?

Is Our University Relation in BCNF?

Relation: R = (InsID, InsName, Office, CID, Title, SID, SName, Major, Grade)
Key: {InsID, CID, SID}

The Test

For every FD $X \rightarrow Y$, is X a superkey of R?

- $InsID \rightarrow InsName, Office$: Is {InsID} a superkey? **No.** It doesn't determine all attributes. VIOLATION!
- $Office \rightarrow InsID$: Is {Office} a superkey? **No.** VIOLATION!
- $CID \rightarrow Title$: Is {CID} a superkey? **No.** VIOLATION!
- $SID \rightarrow SName, Major$: Is {SID} a superkey? **No.** VIOLATION!
- $SID, CID \rightarrow Grade$: Is {SID, CID} a superkey? **No.** VIOLATION!

Is Our University Relation in BCNF?

Relation: R = (InsID, InsName, Office, CID, Title, SID, SName, Major, Grade)
Key: {InsID, CID, SID}

The Test

For every FD $X \rightarrow Y$, is X a superkey of R?

- $InsID \rightarrow InsName, Office$: Is {InsID} a superkey? **No.** It doesn't determine all attributes. VIOLATION!
- $Office \rightarrow InsID$: Is {Office} a superkey? **No.** VIOLATION!
- $CID \rightarrow Title$: Is {CID} a superkey? **No.** VIOLATION!
- $SID \rightarrow SName, Major$: Is {SID} a superkey? **No.** VIOLATION!
- $SID, CID \rightarrow Grade$: Is {SID, CID} a superkey? **No.** VIOLATION!

Conclusion

Our universal relation fails the BCNF test spectacularly. It is full of partial dependencies (where a non-key attribute is determined by part of the key) and transitive dependencies. This is what causes data anomalies.

The BCNF Decomposition Algorithm

The Process

To achieve BCNF, we decompose our relation.

1. Start with a relation R and a set of FDs F.
2. Find a BCNF violation in R, say $X \to Y$.
3. Decompose R into two new relations:
   - $R_1 = XY$ (the attributes in the violating FD)
   - $R_2 = R - Y$ (everything else, but keeping the determinant X)
4. Compute the FDs that hold on R_1 and R_2.
5. Repeat this process on R_1 and R_2 until no BCNF violations remain in any relation.

BCNF Decomposition: Step 1

Relation: R = (InsID, InsName, Office, CID, Title, SID, SName, Major, Grade)
Violation: Let's pick $InsID \rightarrow InsName, Office$.

$R_1 = X \cup Y$

R_1 = (InsID, InsName, Office)

- FDs: $InsID \rightarrow InsName, Office$, $Office \rightarrow InsID$
- Keys: {InsID}, {Office}
- **Is R_1 in BCNF? Yes.** All determinants are keys.

$R_2 = R - Y$

R_2 = (InsID, CID, Title, SID, SName, Major, Grade)

- The attribute 'Office' has been removed.

Next Step

Now we must check R_2 for BCNF violations.

BCNF Decomposition: Step 2

Current Relation: $R_2 =$ (InsID, CID, Title, SID, SName, Major, Grade)
Projected FDs:
$\{CID \rightarrow Title, SID \rightarrow SName, Major, SID, CID \rightarrow Grade\}$
Key of R_2: {InsID, CID, SID}
Violation: Let's pick $CID \rightarrow Title$.

R_{2a}

$R_{2a} =$ (CID, Title)

- FDs: $CID \rightarrow Title$
- Key: {CID}
- **Is it in BCNF? Yes.**

R_{2b}

$R_{2b} =$ (InsID, CID, SID, SName, Major, Grade)

Next Step

We continue this process... The final decomposition will be shown on the next slide.

Final BCNF Decomposition Result

After repeating the process for all violations, we arrive at a well-structured schema:

- **Instructors**: (<u>InsID</u>, InsName, Office)
  - FDs: $InsID \rightarrow InsName, Office$, $Office \rightarrow InsID$
- **Courses**: (<u>CID</u>, Title)
  - FD: $CID \rightarrow Title$
- **Students**: (<u>SID</u>, SName, Major)
  - FD: $SID \rightarrow SName, Major$
- **Enrollment**: (<u>SID, CID</u>, Grade)
  - FD: $SID, CID \rightarrow Grade$
- **Teaches**: (<u>InsID, CID</u>)
  - No FDs here. This table just links instructors to courses.

Final BCNF Decomposition Result

After repeating the process for all violations, we arrive at a well-structured schema:

- **Instructors**: (<u>InsID</u>, InsName, Office)
  - FDs: $InsID \rightarrow InsName, Office$, $Office \rightarrow InsID$
- **Courses**: (<u>CID</u>, Title)
  - FD: $CID \rightarrow Title$
- **Students**: (<u>SID</u>, SName, Major)
  - FD: $SID \rightarrow SName, Major$
- **Enrollment**: (<u>SID, CID</u>, Grade)
  - FD: $SID, CID \rightarrow Grade$
- **Teaches**: (<u>InsID, CID</u>)
  - No FDs here. This table just links instructors to courses.

Observation

This looks exactly like the schema we would have created from the E/R diagram in Lecture 4 if we had followed the standard translation rules! Theory confirms good design principles.

The Two Goals of Decomposition

When we decompose a relation, we want our new schema to have two crucial properties. This is a critical trade-off in database design.

1. Lossless Join

We must be able to reconstruct the original relation by joining the decomposed tables. If we can't, we have lost information. This property is **essential**.

2. Dependency Preservation

All of the original FDs must be enforceable by checking FDs within single relations in the new schema. If an FD spans multiple tables, it's very expensive to check. This property is **highly desirable**.

Lossless Join Property

The Test

A decomposition of R into R_1 and R_2 is lossless if and only if the set of common attributes, $R_1 \cap R_2$, is a **superkey** for either R_1 or R_2.

Lossless Join Property

The Test

A decomposition of R into R_1 and R_2 is lossless if and only if the set of common attributes, $R_1 \cap R_2$, is a **superkey** for either R_1 or R_2.

BCNF Decomposition is Always Lossless

Recall the BCNF algorithm: we decompose R based on a violation $X \to Y$.

- $R_1 = XY$
- $R_2 = R - Y = (R - YX) \cup X$
- The intersection is $R_1 \cap R_2 = X$.
- By definition, $X \to Y$ holds in R_1. So X is a superkey for R_1.
- Therefore, the BCNF decomposition algorithm always produces a lossless join.

Dependency Preservation

The Problem with BCNF

BCNF decomposition is always lossless, but it is **not** always dependency-preserving.

A Classic Example

Consider the relation $R(C, S, Z)$ representing courses, subjects, and zip codes.

- A student takes courses in a certain subject area, and lives in a certain zip code.
- A zip code determines the subject area (e.g., campus zip code determines which department is local).
- FDs:
  - $C, S \rightarrow Z$ (A student in a subject area takes a specific course)
  - $Z \rightarrow S$ (A zip code determines the subject area)
- Keys: $\{C, S\}$ and $\{C, Z\}$.
- BCNF Violation: $Z \rightarrow S$ because Z is not a superkey.

Third Normal Form (3NF): The Compromise

Definition

A relation R is in 3NF if for every non-trivial FD $X \rightarrow Y$, at least one of the following is true:

1. X is a superkey of R, OR
2. Each attribute in Y is part of some candidate key of R.

Third Normal Form (3NF): The Compromise

Definition

A relation R is in 3NF if for every non-trivial FD $X \rightarrow Y$, at least one of the following is true:

1. X is a superkey of R, OR
2. Each attribute in Y is part of some candidate key of R.

Interpretation

3NF relaxes the strict BCNF rule. It allows a non-key to determine another attribute, but *only* if that attribute is "prime" (part of a key). This weakening is precisely what allows us to preserve all dependencies.

Key Idea

Every relation in BCNF is also in 3NF. 3NF is a superset of BCNF.

3NF Synthesis Algorithm

Unlike BCNF's decomposition, 3NF uses a "synthesis" approach that guarantees both lossless joins and dependency preservation.

The Algorithm

1. Find a **minimal cover** G for the set of FDs F. (This means no redundant FDs and no extraneous attributes on the LHS of any FD).
2. For each FD $X \rightarrow Y$ in G, create a relation schema XY.
3. If no relation schema contains a candidate key for the original relation R, add one more schema that consists only of a candidate key.

3NF Synthesis Algorithm

Revisiting the problem case: $R(C, S, Z)$

- FDs: $\{C, S \rightarrow Z, Z \rightarrow S\}$. This is a minimal cover.
- Step 2: Create relations $R_1(C, S, Z)$ from the first FD and $R_2(Z, S)$ from the second. Since R_2 is a subset of R_1, we can discard it.
- We are left with $R_1(C, S, Z)$.
- Step 3: Does R_1 contain a key of the original relation? Yes, $\{C, S\}$ is a key. Done!
- The relation $R(C, S, Z)$ is in 3NF but not BCNF.

BCNF vs. 3NF: The Final Showdown

BCNF

- **Pros:**
  - Purest form.
  - Eliminates all redundancy due to FDs.
  - Conceptually simpler rule (every determinant is a superkey).
- **Cons:**
  - May not preserve all functional dependencies.

3NF

- **Pros:**
  - Always preserves all functional dependencies.
  - Always provides a lossless join.
- **Cons:**
  - May permit some redundancy (as seen in the C,S,Z example).
  - Definition is more complex.

Most practitioners aim for BCNF. If dependency preservation becomes an issue, they may choose 3NF for the problematic relations. Often, dependencies lost under BCNF can be enforced with application logic or triggers if absolutely necessary.

Motivation: When FDs Aren't Enough

Consider this relation describing instructors, the courses they can teach, and the textbooks recommended for those courses.

Course	Instructor	Text
CS101	Smith	DB Systems
CS101	Smith	Ullman
CS101	Jones	DB Systems
CS101	Jones	Ullman

Analysis

- What are the FDs here? **None!** No single attribute determines another.
- The key is {Course, Instructor, Text}. Since the key is the whole relation, it is in BCNF.
- But there is obvious, painful redundancy! To add a new instructor for CS101, we must add two tuples (one for each book).

Multivalued Dependencies (MVDs)

The Problem

The redundancy in the previous example is caused by the fact that for a given course, the set of instructors and the set of textbooks are **independent** of each other.

Definition

A multivalued dependency (MVD) $X \twoheadrightarrow Y$ holds on relation R if for each value of X, the set of values of Y is independent of the values of $R - X - Y$.

Multivalued Dependencies (MVDs)-continued

In our example:

- *Course* $\twoheadrightarrow$ *Instructor*
  - For course 'CS101', the set of instructors {Smith, Jones} is independent of the set of texts.
- *Course* $\twoheadrightarrow$ *Text*
  - For course 'CS101', the set of texts {DB Systems, Ullman} is independent of the set of instructors.

Fourth Normal Form (4NF)

Definition

A relation R is in 4NF if for every non-trivial MVD $X \twoheadrightarrow Y$, X is a superkey for R.

Sound Familiar?

This is the exact same idea as BCNF, but applied to MVDs instead of FDs.

Decomposition for 4NF

Our relation CourseInfo(Course, Instructor, Text) has the MVD $Course \twoheadrightarrow Instructor$, but Course is not a superkey. This is a 4NF violation.

- We decompose into:
  - $R_1(\underline{Course}, Instructor)$
  - $R_2(\underline{Course}, Text)$
- This eliminates the redundancy. Both new relations are in 4NF.

The Ladder of Normalization

5NF (Join Dependencies)
Handles very rare, complex redundancies.

↑

4NF (Multivalued Dependencies)
Every determinant of an MVD is a superkey.

↑

BCNF (Functional Dependencies)
Every determinant of an FD is a superkey.

↑

3NF (Functional Dependencies)
More relaxed; allows some transitive dependencies on prime attributes.

↑

1NF
All attributes are atomic.

Each normal form is a superset of the next. A relation in BCNF is automatically in 3NF and 1NF. Aim for BCNF or 3NF in practice.

Takeaways

- **Normalization** theoretical tool to refine our logical schema, eliminating redundancy and preventing data anomalies.
- **BCNF** is the ideal normal form for FDs, ensuring every determinant is a superkey. It guarantees a lossless join but may not preserve dependencies.
- **3NF** is the pragmatic compromise, guaranteeing both lossless join and dependency preservation, at the cost of allowing some minor redundancy.
- **4NF** addresses more subtle redundancies caused by multivalued dependencies, where attribute sets are independent of each other.